



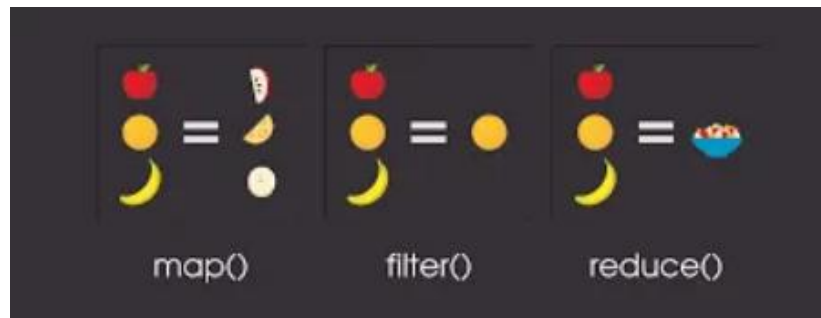
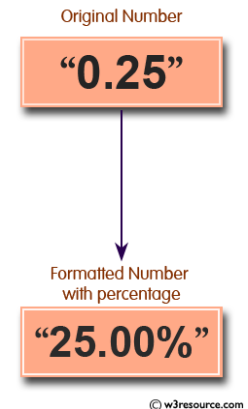
Advanced Python programming using AI tools

Lecture 11: Advanced Programming

Lecturer: Dr. Havana Rika

Today

1. Map Reduce and Filter
2. Formatting Numbers
3. Static\Class Function & Class Variable



Map Reduce and Filter

Map, **filter**, and **reduce** are three powerful functions in Python that allow you to **manipulate** and **transform data** in efficient and concise ways **without** using explicit **loops**.

```
map([🐮, 🍌, 🐔, 🌽], cook)
```

```
=> [🍔, 🍟, 🍗, 🍿]
```

```
filter([🍔, 🍟, 🍗, 🍿], isVegetarian)
```

```
=> [🍟, 🍿]
```

```
reduce([🍔, 🍟, 🍗, 🍿], eat)
```

```
=> 💪
```

Map Function

map(func, iter)

func: It is a function to which map passes each element of given iterable.

iter: iterable object to be mapped.

```
2  def double(n): 1 usage
3      return n * 2
4
5  n = [5, 6, 7, 8]
6  res = map(double, n)
7  print(list(res))
```

[10, 12, 14, 16]

map() function returns a map object(which is an iterator) of the results after **applying** the given **function** to **each item** of a given iterable (list, tuple, etc.).

Map Function

The function returns a **map object**, which is a generator object.

```
19 my_pets = ['alfred', 'tabitha', 'william', 'arla']
20 uppered_pets = map(str.upper, my_pets)
21 print(uppered_pets, type(uppered_pets))
22
23 uppered_pets = list(uppered_pets)
24 print(uppered_pets, type(uppered_pets))
```

```
<map object at 0x0000017887AF16F0> <class 'map'>
['ALFRED', 'TABITHA', 'WILLIAM', 'ARLA'] <class 'list'>
```

```
0 my_pets = ['alfred', 'tabitha', 'william', 'arla']
1 uppered_pets = []
2
3 for pet in my_pets:
4     pet_ = pet.upper()
5     uppered_pets.append(pet_)
6
7 print(uppered_pets)
```

Map Function

`map(func, *iter)`

The number of arguments to func must be the number of iterables listed.

```
circle_areas = [3.56773, 5.57668, 4.00914, 56.24241, 9.01344, 32.00013]

result = list(map(round, circle_areas, range(1, 7)))

print(result)
```

```
[3.6, 5.58, 4.009, 56.2424, 9.01344, 32.00013]
```

```
37 circle_areas = [3.56773, 5.57668, 4.00914, 56.24241, 9.01344, 32.00013]
38
39 result = list(map(round, circle_areas, range(1, 3)))
40
41 print(result)
```

```
[3.6, 5.58]
```

Map Function

map(func, *iter)

The number of arguments to func must be the number of iterables listed.

```
my_strings = ['a', 'b', 'c', 'd', 'e']
my_numbers = [1, 2, 3, 4, 5]

results = list(map(lambda x, y: (x, y), my_strings, my_numbers))

print(results)
```

```
[('a', 1), ('b', 2), ('c', 3), ('d', 4), ('e', 5)]
```

Map Function on Dictionaries

```
data = [{'name': 'Alice', 'age': 30}, {'name': 'Bob', 'age': 25}]
# Get a list of just the names
names = list(map(lambda user: user['name'], data))
print(names) # Output: ['Alice', 'Bob']

# Reverse key-value pairs to create a new dictionary
original_dict = {'a': 1, 'b': 2}
reversed_dict = dict(map(reversed, original_dict.items()))
print(reversed_dict) # Output: {1: 'a', 2: 'b'}
```


Map Function on Dictionaries

```
data = {'apple': 1, 'banana': 2, 'cherry': 3}
# Square each value
squared_values = list(map(lambda x: x * x, data.values()))
print(squared_values)
# Output: [1, 4, 9]
```

```
# Map a list of items to a dictionary with an index as the key
items = ["apple", "banana", "cherry"]
indexed_dict = dict(map(lambda item: (items.index(item), item), items))
print(indexed_dict)
# Output: {0: 'apple', 1: 'banana', 2: 'cherry'}
```

Reduce Function

reduce(func, iter, initial)

func: It is a function **of two arguments cumulatively** to execute on each element of the iterable object.

iter: It is iterable to be reduced

initial (optional): Initial value for the accumulator.

```
11 from functools import reduce
12
13 numbers = [1, 2, 3, 4]
14
15 prod = reduce(lambda x, y: x * y, numbers)
16 print(prod)
```

24

reduce applies a function **of two arguments cumulatively** to the elements of an iterable, optionally starting with an initial argument. The function **reduces iter** into a **single value**

Reduce Function

```
from functools import reduce

lst = [[1,2], [3,4], [5], [6,7,8]]

flattened = reduce(lambda x, y: x + y, lst)
print(flattened)
# Output: [1, 2, 3, 4, 5, 6, 7, 8]

words = ["Hello", "world", "from", "Python"]
sentence = reduce(lambda x, y: x + " " + y, words)
print(sentence)
# Output: "Hello world from Python"
```

Reduce Function

```
from functools import reduce

numbers = [3, 4, 6, 9, 34, 12]

def custom_sum(first, second): 1 usage
    return first + second

result = reduce(custom_sum, numbers)
print(result)
```

68

reduce takes the **first** and **second elements** in **numbers** and passes them to **custom_sum** respectively.

custom_sum computes their **sum** and returns it to **reduce**.

reduce then takes that **result** and applies it as the **first** element to **custom_sum** and takes the **next element** (third) in **numbers** as the **second** element to **custom_sum**.

It does this continuously (cumulatively) until **numbers** is exhausted.

Reduce Function

```
from functools import reduce

numbers = [3, 4, 6, 9, 34, 12]

def custom_sum(first, second): 1 usage
    return first + second

result = reduce(custom_sum, numbers, 10)
print(result)
```

78

The result is 78 because reduce, initially, uses 10 as the first argument to custom_sum.

Filter Function

filter(func, iter)

func: a function that tests if each element of a sequence is true or not.

iter: Iterable which needs to be filtered, it can be sets, lists, tuples, or containers of any iterators.

```
9 def is_even(n): 1 usage
10     return n % 2 == 0
11
12 n = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
13
14 en = filter(is_even, n)
15 print(list(en))
```

```
[2, 4, 6, 8, 10]
```

The filter() method **filters** the given Iterable with the help of a **function** that **tests each element** in the sequence to be **true** or not.

Filter function

```
scores = [66, 90, 68, 59, 76, 60, 88, 74, 81, 65]

def is_A_student(score): 2 usages
    return score > 75

over_75 = list(filter(is_A_student, scores))

print(over_75)
print(type(filter(is_A_student, scores)))
```

```
[90, 76, 88, 81]
<class 'filter'>
```

Filter function

```
words = ("demigod", "rewire", "madam", "freer", "anutforajaroftuna", "kiosk")  
  
palindromes = list(filter(lambda word: word == word[::-1], words))  
  
print(palindromes)
```

```
['madam', 'anutforajaroftuna']
```


Combined Examples

```
19  from functools import reduce
20
21  n = [1, 2, 3, 4, 5, 6]
22
23  e = filter(lambda x: x % 2 == 0, n)
24  s = map(lambda x: x**2, e)
25  res = reduce(lambda x, y: x + y, s)
26
27  print(res)
```

56

Explanation:

filter(lambda x: x % 2 == 0, n): extracts even numbers [2, 4, 6].

map(lambda x: x**2, e): squares each even number and returns [4, 16, 36].

reduce(lambda x, y: x + y, s): sums all squared values $\rightarrow 4 + 16 + 36 = 56$.

Combined Examples

```
40 from functools import reduce
41
42 n = [-3, -1, 2, 4, -2, 5]
43
44 res = reduce(lambda x, y: x * y, map(lambda x: x, filter(lambda x: x > 0, n)))
45
46 print(res)
```

40

Explanation:

filter(lambda x: x > 0, n): selects positive numbers [2, 4, 5].

map(lambda x: x, ...): keeps them unchanged.

reduce(lambda x, y: x * y, ...): multiplies all $\rightarrow 2 \times 4 \times 5 = 40$.

Combined Examples

```
49 from functools import reduce
50
51 w = ["sky", "apple", "tree", "gym", "orange"]
52
53 res = reduce(lambda a, b: a + b, map(lambda w: len(w), filter(lambda w: any(v in w for v in 'aeiou'), w)))
54
55 print(res)
```

15

Explanation:

filter(lambda w: any(v in w for v in 'aeiou'), w): keeps words containing vowels → ['apple', 'tree', 'orange'].

map(lambda w: len(w), ...): converts each to its length [5, 4, 6].

reduce(lambda a, b: a + b, ...): adds all lengths → $5 + 4 + 6 = 15$.

Formatting Numbers

Number	Format	Output	Description
3.1415926	{:.2f}	3.14	2 decimal places
3.1415926	{:+.2f}	+3.14	2 decimal places with sign
-1	{:+.2f}	-1.00	2 decimal places with sign
3.1415926	{:.0f}	3	No decimal places (will round)
5	{:0>2d}	05	Pad with zeros on the left
1000000	{:,}	1,000,000	Number format with comma separator
0.25	{:.2%}	25.00%	Format percentage
1000000000	{:.2e}	1.00e+09	Exponent notation
11	{:>10d}	11	Right aligned
11	{:<10d}	11	Left aligned
11	{:^10d}	11	Center aligned

Displaying Formatted Output with F-strings (1 of 8)

- An f-string is a special type of string literal that is prefixed with the letter `f`

```
>>> print(f'Hello world')  
Hello world
```

- F-strings support placeholders for variables

```
>>> name = 'Johnny'  
>>> print(f'Hello {name}.')  
Hello Johnny.
```

Displaying Formatted Output with F-strings (2 of 8)

- Placeholders can also be expressions that are evaluated

```
>>> print(f'The value is {10 + 2}.')  
The value is 12.
```

```
>>> val = 10  
>>> print(f'The value is {val + 2}.')  
The value is 12.
```

Displaying Formatted Output with F-strings (3 of 8)

- Format specifiers can be used with placeholders

```
>> num = 123.456789
>> print(f'{num:.2f} ')
123.46
>>>
```

- `.2f` means:
 - round the value to 2 decimal places
 - display the value as a floating-point number

Formatting Numbers

```
# This program demonstrates how a floating-point  
# number can be formatted.  
amount_due = 5000.0  
monthly_payment = amount_due / 12  
print ('The monthly payment is',  
      format(monthly_payment, '.2f'))
```

Alternatively, we can write `print(f'The monthly payment is {monthly_payment}.')`



The monthly payment is 416.67

Formatting Numbers

Formatting with Two Decimal Places

value <class 'float'>

123.45678

"{: .2f}".format(value)

appdividend.com

123.46

formatted_value <class 'str'>

Formatting with One Decimal Place

value <class 'float'>

123.45678

"{: .1f}".format(value)

appdividend.com

123.5

formatted_value <class 'str'>



Displaying Formatted Output with F-strings (4 of 8)

- Other examples:

```
>> num = 1000000.00
>> print(f'{num:,.2f} ')
1,000,000.00
```

```
>>> discount = 0.5
>>> print(f'{discount:.0%} ')
50%
```

Displaying Formatted Output with F-strings (5 of 8)

- Other examples:

```
>> num = 123456789
>> print(f'{num:,d} ')
123,456,789
```

```
>>> num = 12345.6789
>>> print(f'{num:.2e} ')
1.23e+04
```

Displaying Formatted Output with F-strings (6 of 8)

- Specifying a minimum field width:

```
>>> num = 12345.6789
>>> print(f'The number is {num:12,.2f}')
```

The number is 12,345.68

Field width = 12

The number is

				1	2	,	3	4	5	.	6	8
--	--	--	--	---	---	---	---	---	---	---	---	---

Field width = 12

Displaying Formatted Output with F-strings (7 of 8)

- Aligning values within a field
 - Use < for left alignment
 - Use > for right alignment
 - Use ^ for center alignment
- Examples:
 - `print(f'{num:<20.2f}')`
 - `print(f'{num:>20.2f}')`
 - `print(f'{num:^20.2f}')`

Displaying Formatted Output with F-strings (8 of 8)

- The order of designators in a format specifier
 - When using multiple designators in a format specifier, write them in this order:

`[alignment][width][,][.precision][type]`

- Example:
 - `print(f'{number:^10,.2f}')`

Displaying Formatted Output with F-strings (8 of 8)

Fill	Width	Grouping	Precision	Type	All Together	Example Output
			.2	f	{num:.2f}	'4125.60'
		,	.2	f	{num:,.2f}	'4,125.60'
0	8		.2	f	{num:08.2f}	'04125.60'
	8		.2	f	{num: 8.2f}	' 4125.60'
			.0	%	{num:.0%}	'50%'

There's also **g**, **G**, **e**, **E**, **n**, and **F** types for floating point numbers and **c**, **o**, and **n** types for integers

Formatting Numbers

- If you want the number to be formatted with comma separators, you can insert a comma into the format specifier, as shown here:

```
# This program demonstrates how a floating-point  
# number can be displayed as currency.  
monthly_pay = 5000.0  
annual_pay = monthly_pay * 12  
print ('Your annual pay is $',  
      format(annual_pay, ',.2f'),  
      sep="")
```



Your annual pay is \$60,000.00



Formatting Numbers (cont'd.)

- The `%` symbol can be used in the format string of `format` function to format number as percentage

```
>>> print(format(0.5, '%'))  
50.000000%
```

- Here is an example that specifies 0 as the precision:

```
>>> print(format(0.5, '.0%'))  
50%
```

```
>>> print(format(0.5, '.1%'))  
50.0%
```

Formatting Numbers (cont'd.)

- To format an integer using `format` function:
 - Use `d` as the type designator
 - Do not specify precision
 - Can still use `format` function to set field width or comma separator

```
>>> print(format(123456, 'd')) ↵  
123456
```

no special formatting

```
>>> print(format(123456, ',d')) ↵  
123,456
```

printed with a comma separator

```
print(format(123456, '10d')) ↵  
123456
```

printed in a field that is 10
spaces wide

```
print(format(123456, '10,d')) ↵  
123,456
```

Printed with a comma separator
in a field that is 10 spaces wide

Formatting Numbers (example)

```
# Example

num1 = 126.844
num2 = 11.147
num3 = 455.821
num4 = 88.081
num5 = 3.776

# Display each number in a field of 7 spaces with 2 decimal places

print(format(num1, '7.2f'))
print(format(num2, '7.2f'))
print(format(num3, '7.2f'))
print(format(num4, '7.2f'))
print(format(num5, '7.2f'))

# Output:
#      126.84
#       11.15
#     455.82
#      88.08
#       3.78
```

Formatting Strings

```
>>> "Hello".format
<built-in method format of str object at 0x7f826da216b0>

>>> "Hello, {}".format("Liam")
'Hello, Liam'

>>> "{} {} {}".format("Hi", "Hello", "How are you")
'Hi Hello How are you'

>>> "{2} {0} {1}".format("Hi", "Hello", "How are you")
'How are you Hi Hello'

>>> "{a} {b} {c}".format(a="a", b="b", c="c")
'a b c'
```

```
1 print(f"Hello")
2
3 name = "Liam"
4 print(f"Hello, {name}")
5
6 x, y, z = "Hi", "Hello", "How are you"
7 print(f"{x} {y} {z}")
8 print(f"{z} | {x} | {y}")
9
10 a, b, c = "aa", "bb", "cc"
11 print(f"{b} {a} {c}")
```

```
Hello
Hello, Liam
Hi Hello How are you
How are you | Hi | Hello
bb aa cc
```

Static Function

A **static method** in Python is a method that belongs to a class but **does not** have **access** to the **instance** (self) or the **class** itself (cls).

It functions like a **regular function** that is **logically grouped** within the **class's namespace** for organization, but operates independently of the class's or instance's state.

Used for a **utility** function.

```
class MyClass: 3 usages
    def __init__(self, value):
        self.value = value

    @staticmethod 2 usages
    def get_max_value(x, y):
        return max(x, y)

# Create an instance of MyClass
obj = MyClass(10)

print(MyClass.get_max_value(x: 20, y: 30))

print(obj.get_max_value(x: 20, y: 30))
```

Static Function

```
1 class PriceTool:
2     @staticmethod
3     def apply_discount(price, rate):
4         """Calculates a discounted price."""
5         return price * (1 - rate)
6
7
8 # Call the static method using the class name
9 original_price = 100
10 discount_rate = 0.2
11 discounted = PriceTool.apply_discount(original_price, discount_rate)
12
13 print(f"The price after discount: ${discounted}")
14 # Output: The price after discount: $80.0
```

Class Variable

```
187 class MyClass: 6 usages
188     # This is a class variable (static variable equivalent)
189     class_counter = 0
190
191     def __init__(self):
192         # Access and modify using the class name
193         MyClass.class_counter += 1
194
195 # Access it using the class name
196 print(f"Initial count: {MyClass.class_counter}")
197
198 # Creating instances increments the shared counter
199 obj1 = MyClass()
200 obj2 = MyClass()
201 obj3 = MyClass()
202
203 # The count is shared among all instances
204 print(f"Count after creating objects: {MyClass.class_counter}")
205 print(f"Count via obj1: {obj1.class_counter}") # Access is possible via instance, but caution is advised
```

```
Initial count: 0
Count after creating objects: 3
Count via obj1: 3
```

Private Class Variable

```
23 class Person: 1 usage
24     __secret_value = 100 # Name mangled
25
26     def get_secret(self): 1 usage
27         return self.__secret_value
28
29 p = Person()
30 print(p.get_secret()) # Works: 100
31 print(p.__secret_value) # Causes an AttributeError
```

100

Traceback (most recent call last):

File "C:\Program Files\JetBrains\PyCharm 2024.1.1\plugins\python-ce\helpers\pydev\pydevconsole.py", line 364, in runcode

coro = func()

^^^^^^

File "<input>", line 9, in <module>

AttributeError: 'Person' object has no attribute '__secret_value'

Private Class Variable

```
class Person: 2 usages
    __secret_value = 100 # Name mangled

    def get_secret(self): 2 usages
        return self.__secret_value

p = Person()
print(p.get_secret()) # Works: 100
print(Person.get_secret()) # Causes an Error
```

100

Traceback (most recent call last):

File "C:\Program Files\JetBrains\PyCharm 2024.1.1\plugins\python-ce\helpers\pydev\pydevconsole.py", line 364, in runcode

```
    coro = func()
    ^^^^^
```

File "<input>", line 9, in <module>

TypeError: Person.get_secret() missing 1 required positional argument: 'self'

Private Class Variable

```
221     class Person: 1 usage
222         __secret_value = 100 # Name mangled
223
224         @staticmethod 1 usage
225         def get_secret():
226             return __secret_value
227
228     print(Person.get_secret()) # Causes an Error
```

```
Traceback (most recent call last):
  File "C:\Program Files\JetBrains\PyCharm 2024.1.1\plugins\python-ce\helpers\pydev\pydevconsole.py", line 364, in runcode
    coro = func()
           ^^^^^^
  File "<input>", line 8, in <module>
  File "<input>", line 6, in get_secret
NameError: name '_Person__secret_value' is not defined
```

Class Method

A class method is a **method** that is **bound** to the **class** and not the object of the class.

A **class method receives** the **class** as an implicit first argument, just like an instance method receives the instance.

```
class Person: 2 usages
    __secret_value = 100 # Name mangled

    @classmethod 2 usages
    def get_secret(cls):
        return cls.__secret_value

p = Person()
print(p.get_secret()) # Works: 100
print(Person.get_secret()) # Works: 100
```

100

100

```
from datetime import date
```

```
class Person: 3 usages
```

```
def __init__(self, name, age):  
    self.name = name  
    self.age = age
```

```
# a class method to create a Person object by birth year.
```

```
@classmethod 1 usage
```

```
def fromBirthYear(cls, name, year):  
    return cls(name, date.today().year - year)
```

```
@staticmethod 1 usage
```

```
def isAdult(age):  
    return age > 18
```

```
person1 = Person(name='mayank', age=21)
```

```
person2 = Person.fromBirthYear(name='mayank', year=1996)
```

```
print(person1.age)
```

```
print(person2.age)
```

```
print(Person.isAdult(22))
```

Class Method

21

30

True